

Systems, Drawn · Video #4

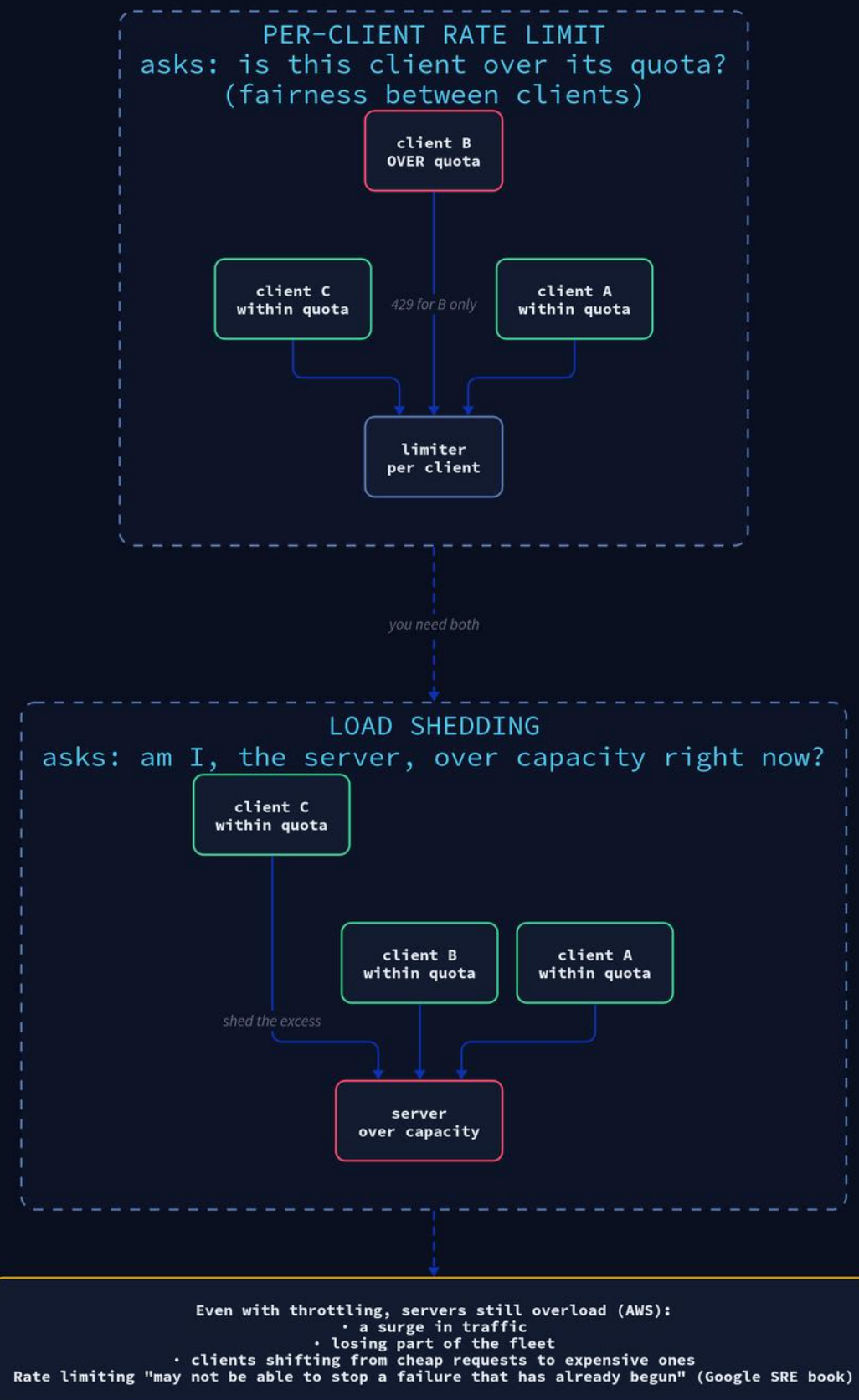
Rate limiting isn't enough: load shedding explained

Diagram pack · CC BY 4.0 · SRE // microservices

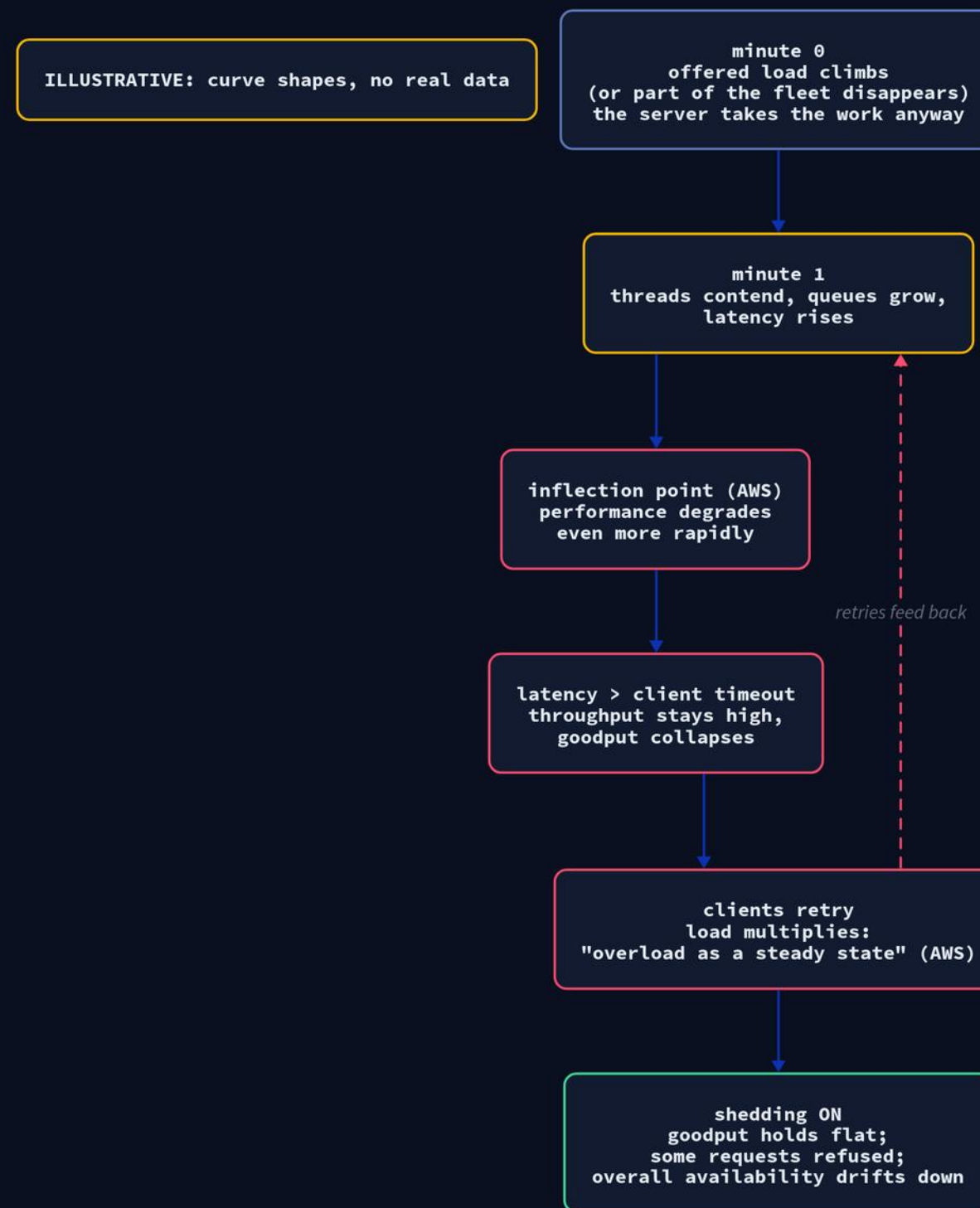
1. Per-client rate limit vs load shedding
2. An overload, over time (illustrative)
3. Failure dynamics: the whole overload in 6 steps
4. Inside one service: priority, queue, limit
5. Never shed the health check
6. How to queue: FIFO trap, LIFO + deadline
7. How much to accept: an adaptive concurrency limit
8. Where to shed: as early as you can
9. What load shedding does NOT solve
10. Neighbor 1: bulkheads (isolation, not shedding)
11. Neighbor 2: hedged requests add load

Sources: AWS Builders' Library (Yanacek, 2019) · Google SRE book ch. 21-22 · Netflix Tech Blog (2020, 2024) · Netflix concurrency-limits · Envoy docs · RFC 8289 ·

1. Per-client rate limit vs load shedding



2. An overload, over time (illustrative)

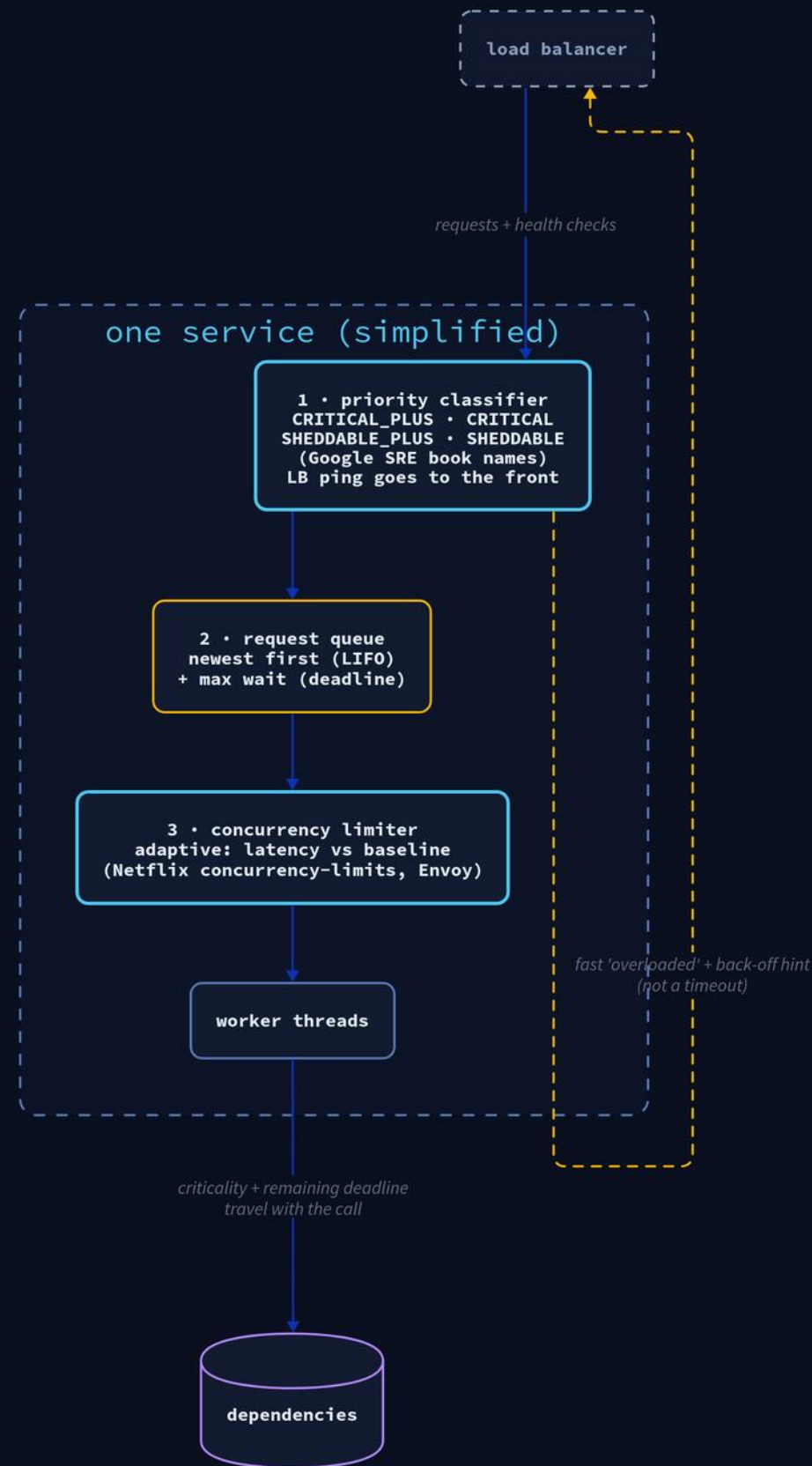


throughput = every request sent to the server
goodput = requests handled without errors, fast enough for the client to use (AWS)
Google's example: healthy at 10,000 QPS, crashing at 11,000 QPS;
dropping to 9,000 QPS will almost certainly not stop the crashes

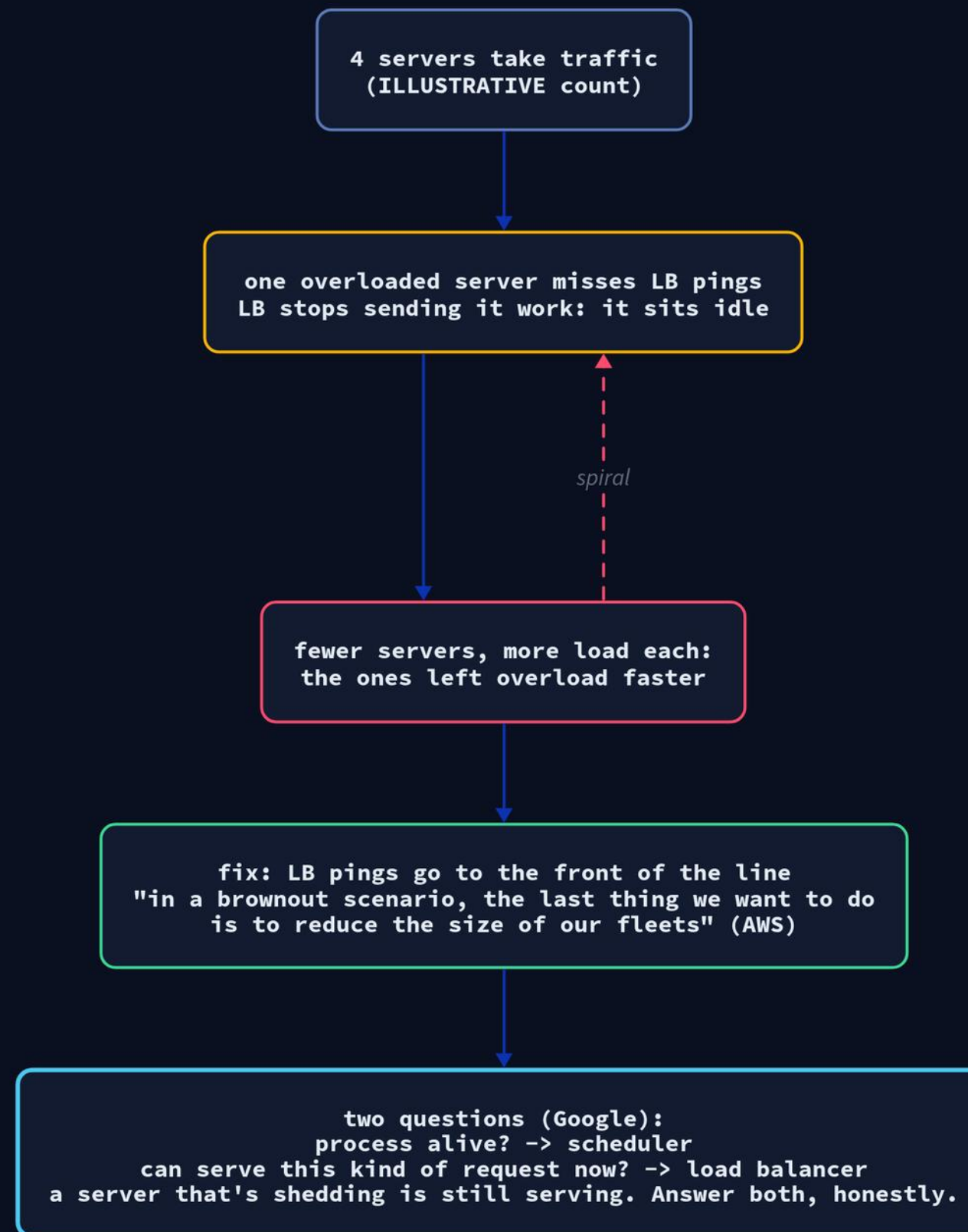
3. Failure dynamics: the whole overload in 6 steps

stage	what happens (generic server, illustrative)
1 • first event	traffic climbs, or capacity drops (surge, lost fleet, cheap-to-expensive requests)
2 • signal	latency crosses the client's timeout: throughput stays high, goodput collapses
3 • shared resource	threads and queues fill; retries multiply the offered load
4 • containment	shed by priority, newest first, within an adaptive limit, as early as you can
5 • trade-off	some requests are refused on purpose; overall availability drifts down
6 • recovery	goodput holds, then load ramps back gradually

4. Inside one service: priority, queue, limit



5. Never shed the health check



6. How to queue: FIFO trap, LIFO + deadline

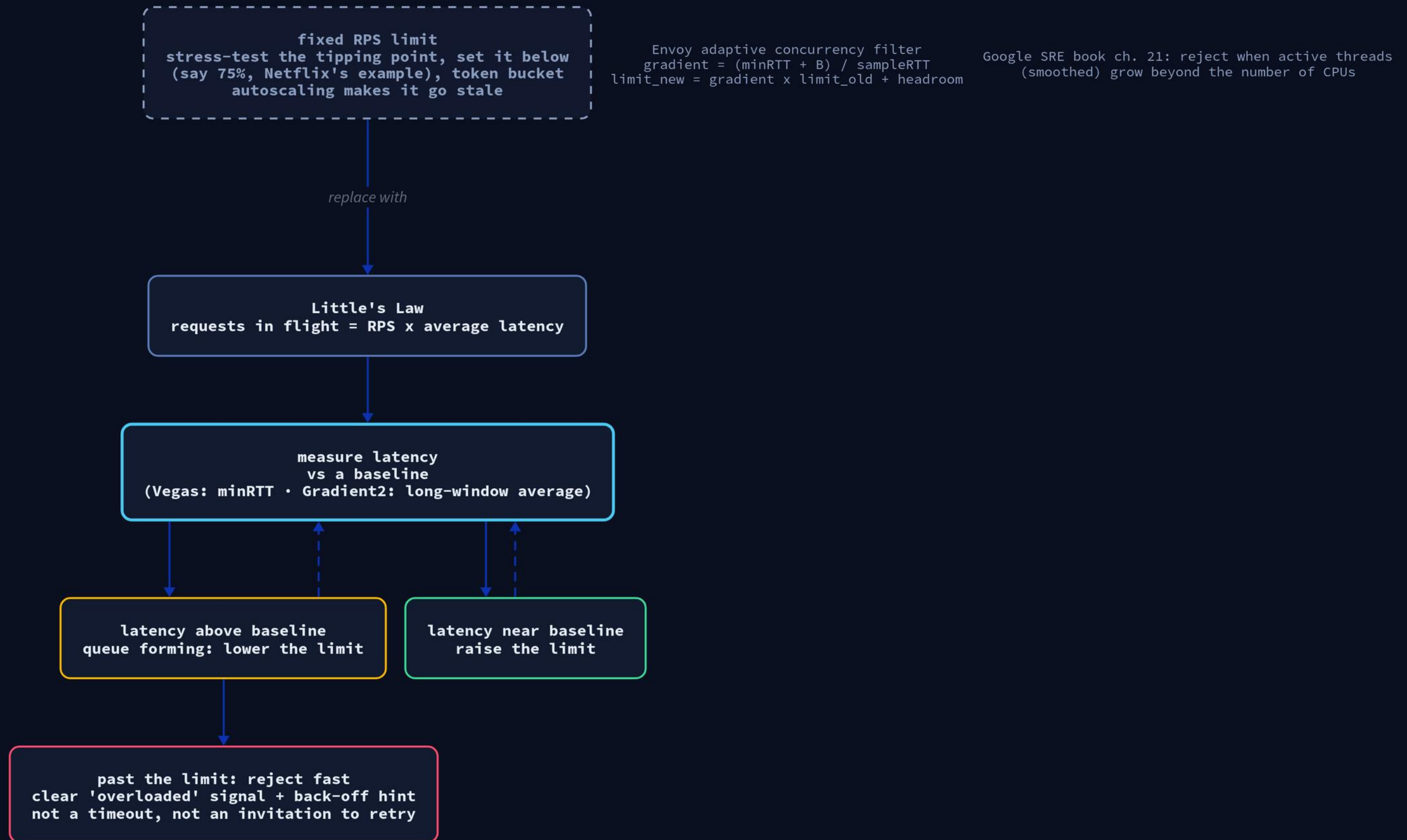
FIFO under overload (Google's example)
queue = 10x threads · 100 ms per request
a request joining a full queue takes 1.1 s,
most of it waiting
client timeout 1 s (ILLUSTRATIVE):
the reply arrives after the client gave up

LIFO + a deadline
serve the newest request first (its client is most likely still waiting)
throw out anything older than the max wait (AWS)
CoDel: built for network packets (RFC 8289);
on request queues it's an adaptation: watch time in queue

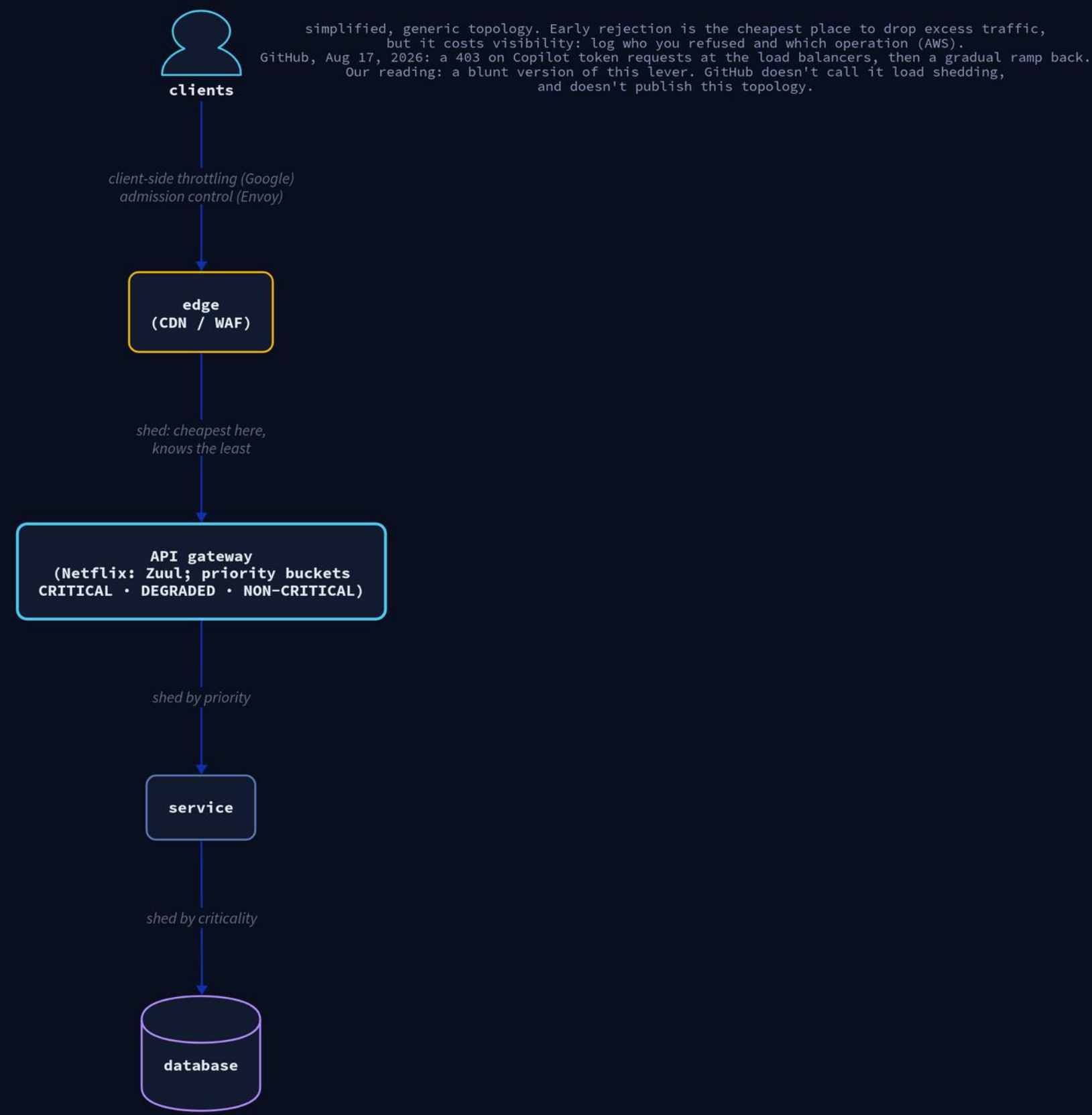
cost of LIFO: fairness
the oldest requests can wait forever
(our reading), so always pair it with a deadline

deadline propagation (Google's example)
A picks 30 s, spends 7 s,
calls B with 23 s left

7. How much to accept: an adaptive concurrency limit



8. Where to shed: as early as you can



9. What load shedding does NOT solve

does NOT solve

why / what to do instead

1 • no new capacity

in a brownout, the priority is to add capacity
and address the bottleneck (AWS)

2 • can blind your autoscaler

shed at about the same CPU target that triggers scaling
and the signal never comes, or comes late (AWS).
Netflix (2024): shed only above the autoscaling CPU target

3 • can flatter your dashboards

fast rejections drag latency down: shed 60% and the median
might look pretty amazing while successful requests are terrible (AWS)

4 • your priority labels can be wrong

Netflix: a low-priority request, when shed, triggered a race condition
on Android and iOS (sporadic playback errors). Test shedding on purpose

5 • clients that ignore the signal

a refused client that retries at once comes back:
client-side retry budgets + backoff with jitter

neighbors

bulkheads are isolation, not shedding;
hedged requests add load (hedge with headroom, shed without)

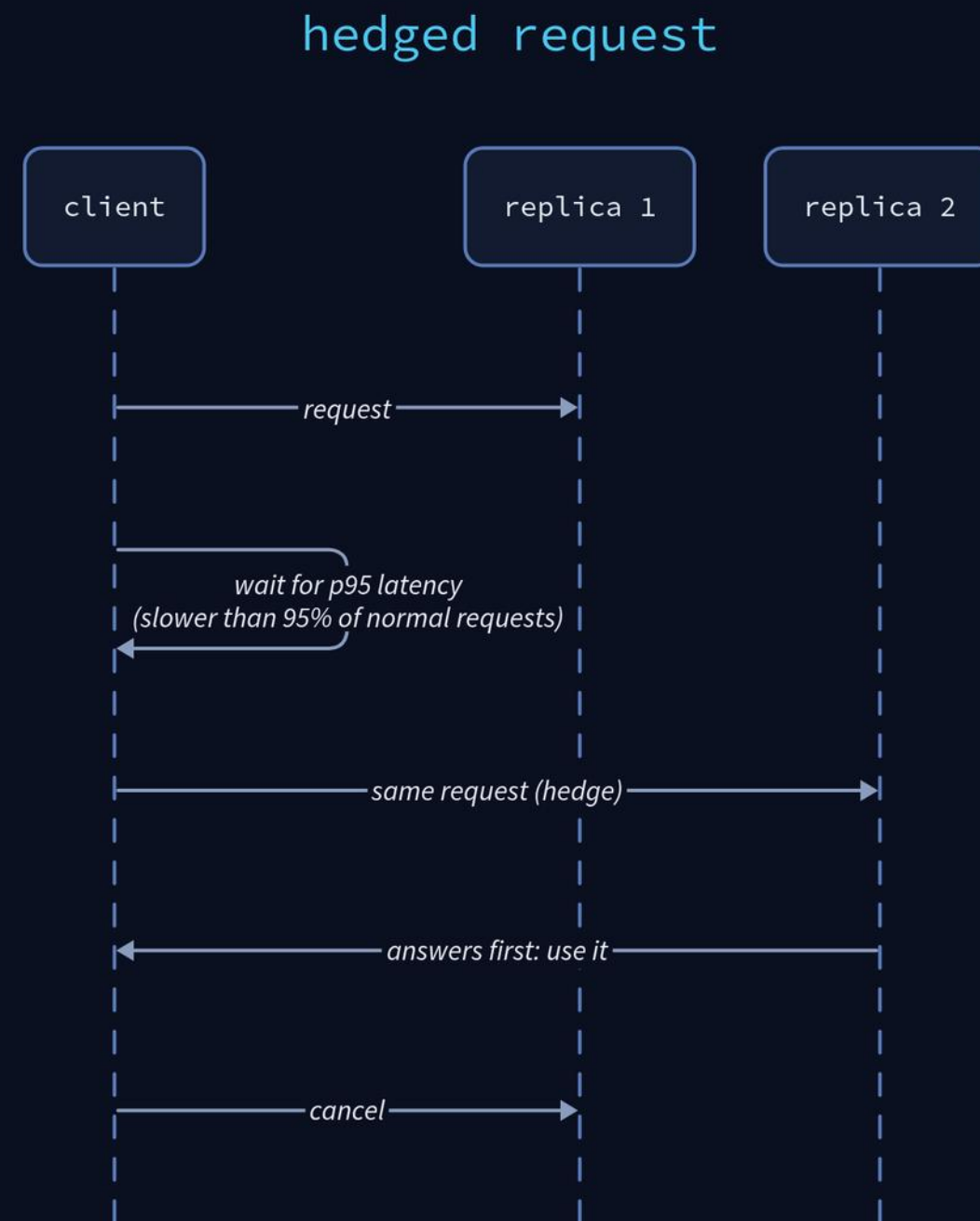
10. Neighbor 1: bulkheads (isolation, not shedding)

Google's suggestion
cap any one client at 25% of threads

Netflix concurrency-limits
live traffic 90% • batch 10%

isolation is not shedding
if everyone is legitimately busy,
every compartment fills up
and you still choose what to drop

11. Neighbor 2: hedged requests add load



Waiting for the p95 before hedging limits the extra load to about 5%.
Google benchmark (2013): p99.9 1,800 ms to 74 ms with +2% requests
(hedge after 10 ms, 1,000 keys on 100 servers).
Our rule: hedge when you have headroom, shed when you don't.